

Transport Layer Study Guide

Questions & Answers

Page 1-4: Introduction & Transport-Layer Services

Question 1

What is the primary role of the transport layer?

To provide communication services directly to application processes running on different hosts.

Question 2

Which two Internet transport-layer protocols are given particular emphasis in this chapter?

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol).

Question 3

What fundamental problem does the transport layer solve regarding data communication?

How two entities can communicate reliably over a medium that may lose and corrupt data.

Question 4

Besides reliable data transfer, what other fundamental problem in networking does the transport layer address?

Controlling the transmission rate of transport-layer entities to avoid or recover from congestion within the network.

Question 5

What is meant by "logical communication" at the transport layer?

From an application's perspective, it is as if the hosts running the processes are directly connected, even though they may be physically far apart and connected via complex network infrastructure.

Page 5-6: Transport-Layer Functionality

Question 6

What does the transport layer convert application-layer messages into?

Transport-layer packets, known as segments in Internet terminology.

Question 7

How do network routers interact with the transport-layer segment within a datagram?

They do not examine the fields of the transport-layer segment; they act only on the network-layer fields of the datagram.

Question 8

On the receiving side, which layer extracts the transport-layer segment from the network-layer datagram?

The network layer.

Page 7-10: Relationship Between Transport and Network Layers

Question 9

What is the key difference between the logical communication provided by the transport layer versus the network layer?

The transport layer provides logical communication between processes, while the network layer provides logical communication between hosts.

Question 10

Where do transport-layer protocols reside?

In the end systems.

Question 11

Do intermediate routers act on information added by the transport layer?

No, they neither act on nor recognize any information added by the transport layer.

Question 12

How can the service model of the underlying network-layer protocol constrain a transport-layer protocol?

If the network layer cannot provide delay or bandwidth guarantees, then the transport layer cannot provide those guarantees to the application.

Question 13

Give an example of a service a transport protocol can provide even if the underlying network layer does not.

Reliable data transfer, even over an unreliable network layer that loses, garbles, or duplicates packets.

Question 14

What are the two distinct transport-layer protocols available in the Internet?

UDP (User Datagram Protocol) and TCP (Transmission Control Protocol).

Question 15

What type of service does UDP provide?

An unreliable, connectionless service.

Question 16

What type of service does TCP provide?

A reliable, connection-oriented service.

Question 17

What is the service model of the Internet's network-layer protocol (IP)?

Best-effort delivery service.

Question 18

Is IP a reliable or unreliable service? Justify your answer.

Unreliable. It does not guarantee segment delivery, orderly delivery, or the integrity of the data.

Question 19

What is the most fundamental responsibility shared by both UDP and TCP?

To extend IP's host-to-host delivery service to a process-to-process delivery service (multiplexing and demultiplexing).

Question 20

What are the only two services that UDP provides?

Process-to-process data delivery and error checking.

Question 21

List at least three additional services that TCP provides beyond what UDP offers.

Reliable data transfer, flow control, and congestion control.

Question 22

What does TCP congestion control prevent?

It prevents any one TCP connection from swamping the links and routers between communicating hosts with an excessive amount of traffic.

Page 16-22: Multiplexing and Demultiplexing (General)

Question 23

What is transport-layer demultiplexing?

The job of delivering the data in a transport-layer segment to the correct socket at the receiving host.

Question 24

What is transport-layer multiplexing?

The job of gathering data chunks from different sockets at the source host, encapsulating each with header information to create segments, and passing the segments to the network layer.

Question 25

To what does the transport layer deliver data directly on the receiving host?

To an intermediary socket, not directly to the process.

Question 26

What two special fields in the transport-layer segment are used for demultiplexing?

The source port number field and the destination port number field.

Question 27

What is the size of a port number field?

16 bits.

Question 28

What is the range of well-known port numbers?

0 to 1023.

Page 23-25: Connectionless (UDP) Multiplexing and Demultiplexing

Question 29

In UDP, how is a socket fully identified?

By a two-tuple: (Destination IP Address, Destination Port Number).

Question 30

In the context of UDP, what is the purpose of the source port number in a segment?

It serves as part of a "return address"; the response will use it as the destination port.

Page 26-30: Connection-Oriented (TCP) Multiplexing and Demultiplexing

Question 31

In TCP, how is a socket fully identified?

By a four-tuple: (Source IP Address, Source Port Number, Destination IP Address, Destination Port Number).

Question 32

How can a server distinguish between two separate HTTP connections from the same host?

The connections will have different source port numbers on the client host.

Question 33

How can a server distinguish between two separate HTTP connections from two different hosts that happen to use the same source port number?

The connections will have different source IP addresses.

Question 34

What is a common modern approach for high-performing Web servers to handle multiple client connections?

Using one process and creating a new thread with a new connection socket for each new client connection.

Question 35

How does the use of non-persistent HTTP impact socket management on a Web server?

A new TCP connection and socket are created and closed for every request/response, which can impact performance.

Page 32-34: UDP Basics

Question 36

What does UDP add to the basic network-layer service?

Multiplexing/demultiplexing and some light error checking.

Question 37

Why is UDP said to be connectionless?

Because there is no handshaking between sending and receiving transport-layer entities before sending a segment.

Question 38

Name an application-layer protocol that typically uses UDP.

DNS (Domain Name System).

Page 35-41: Why Use UDP?

Question 39

List three reasons why an application developer might choose UDP over TCP.

1. Finer application-level control over what data is sent and when.
 2. No connection establishment delay.
 3. No connection state, allowing a server to support more active clients.
 4. Smaller packet header overhead (8 bytes vs. 20).
-

Question 40

Why is TCP's service model not well-matched to the needs of real-time applications?

Because its congestion control can throttle the sender and introduce delay, and its reliable data transfer can cause significant delays due to retransmissions, whereas real-time applications can tolerate some loss but require low delay.

Question 41

Why does DNS typically run over UDP rather than TCP?

To avoid the connection-establishment delay introduced by TCP, which would make DNS lookups much slower.

Question 42

What is a significant problem with running multimedia applications over UDP?

UDP has no congestion control, which can lead to network congestion and "crowding out" of TCP sessions.

Question 43

Is it possible to have reliable data transfer when using UDP? How?

Yes, by building reliability (e.g., acknowledgment and retransmission mechanisms) into the application itself.

Page 42-45: UDP Segment Structure & Checksum

Question 44

What are the four fields in the UDP header?

Source Port Number, Destination Port Number, Length, and Checksum.

Question 45

What is the purpose of the UDP length field?

It specifies the number of bytes in the entire UDP segment (header plus data).

Question 46

Why does UDP provide a checksum when many link-layer protocols also provide error checking?

Because there is no guarantee that all links between source and destination provide error checking, and errors could also be introduced when a segment is stored in a router's memory. Thus, end-to-end error detection is necessary.

Question 47

What does UDP typically do when it receives a segment with an error?

Some implementations discard the segment; others pass it to the application with a warning.

Page 46-50: Principles of Reliable Data Transfer (Introduction)

Question 48

What is the service abstraction provided by a reliable data transfer protocol?

A reliable channel through which data is transferred without corruption, loss, and in order.

Question 49

What are the names of the key procedures invoked on the sending and receiving sides of the rdt protocol?

rdt_send(), udt_send(), rdt_rcv(), deliver_data().

Question 50

In this chapter's development of reliable data transfer, what assumption is made about packet delivery order?

Packets are delivered in the order in which they were sent, though some may be lost.

Page 51-54: rdt1.0

Question 51

What underlying channel characteristic is assumed by rdt1.0?

A perfectly reliable channel.

Question 52

How many states do the sender and receiver FSMs have in rdt1.0?

One state each.

Page 55-60: rdt2.0

Question 53

What new channel impairment does rdt2.0 introduce?

Bit errors in packets.

Question 54

What are the three additional protocol capabilities required for ARQ protocols?

Error detection, receiver feedback (ACKs/NAKs), and retransmission.

Question 55

What are the two types of feedback messages used in rdt2.0?

Positive Acknowledgments (ACKs) and Negative Acknowledgments (NAKs).

Question 56

What is a stop-and-wait protocol?

A protocol where the sender sends one packet and then waits for an acknowledgment before sending the next.

Question 57

In rdt2.0, can the sender get new data from the upper layer while in the "wait for ACK or NAK" state?

No.

Page 61-69: rdt2.1 & rdt2.2

Question 58

What is the fatal flaw in rdt2.0?

It does not account for the possibility that ACK or NAK packets could themselves be corrupted.

Question 59

What mechanism is introduced in rdt2.1 to handle the flaw of corrupted ACK/NAK packets?

A sequence number (a 1-bit sequence number suffices for a stop-and-wait protocol).

Question 60

How does a 1-bit sequence number help the receiver identify a retransmission?

The receiver can check if the sequence number of the received packet is the same as the most recently received packet (retransmission) or has changed (new packet).

Question 61

What is the key difference between rdt2.1 and rdt2.2?

rdt2.2 is NAK-free; it uses duplicate ACKs (ACKing the last correctly received packet) to signal that a packet was received in error.

Question 62

In rdt2.2, what does a sender interpret when it receives two ACKs for the same packet?

It knows that the receiver did not correctly receive the packet following the one that is being ACKed twice.

Page 70-76: rdt3.0

Question 63

What two new concerns must be addressed when the underlying channel can lose packets?

How to detect packet loss and what to do when it occurs.

Question 64

How does rdt3.0 detect packet loss?

By using a countdown timer. If an ACK is not received before the timer expires, the packet is assumed lost.

Question 65

What action does the sender take when a timeout occurs in rdt3.0?

It retransmits the packet and restarts the timer.

Question 66

What problem is introduced if the timeout value is too small? Too large?

Too small: Unnecessary retransmissions. Too large: Slow to react to actual packet loss, increasing delay.

Question 67

How does the use of sequence numbers in rdt2.2 help rdt3.0 handle duplicate packets caused by premature timeouts?

The receiver can use the sequence number to detect and discard duplicate packets.

Question 68

What is another name for the rdt3.0 protocol?

The alternating-bit protocol.

Page 77-84: Pipelined Reliable Data Transfer

Question 69

What is the main performance problem with stop-and-wait protocols like rdt3.0?

Low sender utilization, as the sender is idle for a long time waiting for ACKs.

Question 70

How is sender utilization defined?

The fraction of time the sender is actually busy sending bits into the channel.

Question 71

What is the solution to the poor performance of stop-and-wait protocols?

Pipelining: allowing the sender to send multiple packets without waiting for acknowledgments.

Question 72

What are the three consequences of pipelining for reliable data transfer protocols?

1. The range of sequence numbers must be increased.
 2. The sender and receiver must be able to buffer multiple packets.
 3. The protocol needs a policy for handling errors (Go-Back-N or Selective Repeat).
-

Page 85-97: Go-Back-N (GBN)

Question 73

In GBN, what is the maximum number of unacknowledged packets allowed in the pipeline?

N (the window size).

Question 74

What are the four intervals of sequence numbers in the GBN sender's view?

$[0, \text{base}-1]$ (ACK'd), $[\text{base}, \text{nextseqnum}-1]$ (sent, not ACK'd), $[\text{nextseqnum}, \text{base}+N-1]$ (usable), $[\text{base}+N, \dots]$ (not usable).

Question 75

What is another name for the GBN protocol?

A sliding-window protocol.

Question 76

What are the three types of events the GBN sender must respond to?

Invocation from above (`rdt_send`), receipt of an ACK, and a timeout event.

Question 77

In GBN, what does an acknowledgment for sequence number n represent?

A cumulative acknowledgment, meaning all packets up to and including n have been correctly received.

Question 78

How does the GBN sender handle a timeout event?

It retransmits all packets that have been previously sent but not yet acknowledged. This is the "Go-Back-N" behavior.

Question 79

How many timers does the GBN sender use?

A single timer for the oldest transmitted but not yet acknowledged packet.

Question 80

When does the GBN receiver send an ACK for a packet with sequence number n ?

Only if it is received correctly and in order (i.e., it is the expected packet).

Question 81

What does the GBN receiver do with an out-of-order, but error-free, packet?

It discards it and resends an ACK for the last correctly received in-order packet.

Question 82

What is the main advantage of the GBN receiver discarding out-of-order packets?

Simplicity; the receiver does not need to buffer any out-of-order packets.

Page 98-100: Selective Repeat (SR)

Question 83

What is the main drawback of GBN in environments with high error rates and large bandwidth-delay products?

A single packet error can cause GBN to retransmit a large number of packets, many unnecessarily, wasting bandwidth.

Question 84

How does Selective Repeat (SR) avoid unnecessary retransmissions?

The sender retransmits only those packets that are suspected to be lost or corrupted.

Question 85

How does SR require the receiver to acknowledge packets?

It must individually acknowledge correctly received packets, whether they are in order or not.

Question 86

How does an SR receiver handle out-of-order packets?

It buffers (saves) them until the missing packets are received, then delivers a contiguous range of data to the upper layer.

Page 101-109: TCP Connection Basics

Question 87

Why is TCP called a connection-oriented protocol?

Because the two processes must first "handshake" with each other to establish the parameters of the data transfer before sending data.

Question 88

Do intermediate network elements (routers) maintain TCP connection state?

No, TCP connection state is maintained only in the end systems.

Question 89

What two services does a TCP connection provide?

Full-duplex service and point-to-point service.

Question 90

What is the name of the procedure used to establish a TCP connection?

The three-way handshake.

Question 91

Where does TCP place data received from an application process before sending it?

In the connection's send buffer.

Question 92

How is the Maximum Segment Size (MSS) typically determined?

It is set to ensure that a TCP segment (plus TCP/IP headers) fits into a single link-layer frame (the MTU). A common MSS is 1460 bytes for an Ethernet MTU of 1500.

Question 93

What does the MSS refer to?

The maximum amount of application-layer data in the segment, not the total segment size.

Page 110-114: TCP Segment Structure

Question 94

What is the typical size of the TCP header?

20 bytes (without options).

Question 95

List the key fields in a TCP segment header.

Source/Dest Port, Sequence Number, Acknowledgment Number, Receive Window, Header Length, Checksum, Flag Bits (URG, ACK, PSH, RST, SYN, FIN), and Options.

Question 96

What is the purpose of the receive window field?

It is used for flow control, indicating the number of bytes the receiver is willing to accept.

Question 97

Which flag bits are used for connection setup and teardown?

SYN, FIN, and RST.

Question 98

What does the ACK bit being set to 1 indicate?

That the value in the acknowledgment field is valid.

Page 115-119: TCP Sequence & ACK Numbers

Question 99

How is the sequence number for a segment determined in TCP?

It is the byte-stream number of the first byte in the segment's data field.

Question 100

How is the acknowledgment number in a TCP segment determined?

It is the sequence number of the next byte that the host expects to receive from the other side (cumulative acknowledgment).

Question 101

What does a cumulative acknowledgment mean?

That the receiver is acknowledging all bytes up to, but not including, the acknowledgment number.

Question 102

According to the text, what is the more efficient approach for a TCP receiver when it receives out-of-order segments?

To buffer the out-of-order bytes and wait for the missing bytes to fill in the gaps.

Page 120-125: TCP Round-Trip Time Estimation & Timeout

Question 103

What is the SampleRTT?

The amount of time between when a segment is sent and when its acknowledgment is received.

Question 104

Does TCP compute a SampleRTT for retransmitted segments?

No, it only measures SampleRTT for segments transmitted once.

Question 105

What is the formula for updating EstimatedRTT?

EstimatedRTT = $(1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$ (with α typically 0.125).

Question 106

What is DevRTT an estimate of?

How much the SampleRTT typically deviates from the EstimatedRTT.

Question 107

What is the formula for TimeoutInterval in TCP?

TimeoutInterval = EstimatedRTT + 4 * DevRTT.

Page 126-143: TCP Reliable Data Transfer

Question 108

How many retransmission timers does the recommended TCP timer management procedure use?

A single retransmission timer.

Question 109

What are the three major events that the TCP sender must handle?

Data received from application, timer timeout, and ACK receipt.

Question 110

With which segment is the timer typically associated?

The oldest unacknowledged segment.

Question 111

In Figure 3.34, why does Host B discard the bytes in the retransmitted segment?

Because it has already received that data (bytes 92-99) and delivered it to the application.

Question 112

How does a cumulative acknowledgment (as in Figure 3.36) help avoid unnecessary retransmissions?

An ACK for a later byte implicitly acknowledges all earlier bytes. If an ACK for an earlier segment is lost, but a cumulative ACK for a later segment arrives, the sender knows the earlier data was received and doesn't retransmit it.

Question 113

What is the modification made to the timeout interval after a timer expiration?

The timeout interval is doubled (exponential backoff) after each retransmission of the same segment.

Question 114

What is the primary reason for this exponential backoff?

It is a form of congestion control, as a timeout is likely due to network congestion.

Question 115

What is a duplicate ACK?

An ACK that re-acknowledges a segment for which the sender has already received an earlier acknowledgment.

Question 116

How does a TCP receiver generate duplicate ACKs?

When it receives an out-of-order segment, it sends an ACK for the last in-order byte it received.

Question 117

What is fast retransmit?

If the TCP sender receives three duplicate ACKs for the same data, it interprets this as an indication that the segment following the ACKed segment was lost and retransmits that one segment immediately, without waiting for a timeout.

Question 118

Why is TCP's error-recovery mechanism considered a hybrid of GBN and SR?

Like GBN, it uses cumulative ACKs and a single retransmission timer. Like SR, it often buffers out-of-order segments and can use fast retransmit to retransmit only a single lost segment.

Page 144-150: TCP Flow Control

Question 119

What is the goal of TCP flow control?

To prevent the sender from overflowing the receiver's buffer.

Question 120

How is flow control different from congestion control?

Flow control is a receiver-to-sender mechanism to prevent buffer overflow, while congestion control is a network-to-sender mechanism to prevent network overload.

Question 121

What variable does the TCP receiver use to inform the sender of its available buffer space?

The receive window (rwnd).

Question 122

How is rwnd calculated?

$$\text{rwnd} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

Question 123

How does the sender ensure it does not overflow the receiver's buffer?

It ensures that the amount of unacknowledged data ($\text{LastByteSent} - \text{LastByteAcked}$) is less than or equal to the receiver's advertised rwnd.

Question 124

What problem occurs if the receiver advertises a rwnd of zero?

The sender will stop sending data. If the receiver later has space but has no data/ACK to send, the sender will never find out, causing a deadlock.

Question 125

How is this deadlock problem solved?

The TCP specification requires the sender to continue sending segments with one data byte periodically when the receiver's window is zero. These "probe" segments will trigger ACKs containing the updated (non-zero) rwnd.

Question 126

Does UDP provide flow control?

No.

Page 151-162: TCP Connection Management

Question 127

What are the three steps in the TCP three-way handshake?

1. Client sends a SYN segment (SYN=1) with initial sequence number (client_isn).
 2. Server sends a SYNACK segment (SYN=1, ACK=1) with its own initial sequence number (server_isn) and an acknowledgment number = client_isn + 1.
 3. Client sends an ACK segment (SYN=0, ACK=1) with acknowledgment number = server_isn + 1.
-

Question 128

What is the purpose of the initial sequence number (ISN)? Why is it randomized?

It ensures that each connection starts with a fresh sequence space. Randomization is for security, to avoid certain attacks where an attacker guesses the sequence number.

Question 129

What flag bits are set in the initial SYN segment from the client?

The SYN bit is set to 1.

Question 130

What flag bits are set in the server's SYNACK segment?

The SYN and ACK bits are set to 1.

Question 131

What does the client's sequence number in the third step of the handshake signify?

The SYN bit is 0, indicating that this segment is part of the established connection and the sequence number reflects the first byte of data (if any) in this segment (client_isn+1).

Question 132

How is a TCP connection torn down?

Typically, each side closes its side of the connection independently using a four-step process involving FIN segments.

Question 133

What does a host send to initiate connection teardown?

A TCP segment with the FIN bit set to 1.

Question 134

What is the purpose of the TIME_WAIT state?

To allow the TCP client to resend the final ACK in case it is lost, ensuring a reliable connection teardown.

Question 135

What state does the client TCP enter after sending its FIN segment?

FIN_WAIT_1.

Question 136

What state does the server TCP enter after receiving a FIN and sending its ACK?

CLOSE_WAIT.

Question 137

What state does the server TCP enter after sending its own FIN?

LAST_ACK.

Question 138

What does a host send when it receives a TCP segment for a port that is not accepting connections?

A reset segment (RST flag bit set to 1).

Question 139

What is the initial state of a TCP connection on both client and server?

CLOSED.

Question 140

What state is the server in when it is waiting for a connection request?

LISTEN.

Question 141

What is the state of a TCP connection when data transfer can occur?

ESTABLISHED.

Question 142

Describe the typical state sequence for a client TCP: CLOSED -> ? -> ? -> ESTABLISHED -> ...

CLOSED → SYN_SENT → ESTABLISHED.

Question 143

What state comes after FIN_WAIT_1 for the client if it receives an ACK for its FIN?

FIN_WAIT_2.

Question 144

What is the final state a client TCP passes through before returning to CLOSED?

TIME_WAIT.

Comprehensive & Cross-Topic Questions

Question 145

Compare and contrast the service models of TCP and UDP.

TCP provides a reliable, connection-oriented service with flow control, congestion control, and in-order delivery. UDP provides an unreliable, connectionless service with only multiplexing/demultiplexing and error checking.

Question 146

Explain how multiplexing/demultiplexing works in UDP versus TCP.

UDP uses a two-tuple (dest IP, dest port) to direct a segment to a socket. TCP uses a four-tuple (source IP, source port, dest IP, dest port) to uniquely identify a connection socket.

Question 147

Describe the journey of a data byte from an application process on one host to an application process on another host, focusing on the transport layer.

The sending process passes data to its socket. The transport layer (TCP/UDP) on the sending host encapsulates the data in a segment (adding headers for multiplexing, etc.). This segment is passed to the network layer. On the receiving host, the network layer passes the segment to the transport layer, which demultiplexes it to the correct socket based on port numbers (and IP addresses for TCP). The data is then read by the receiving application process from its socket.

Question 148

How does TCP transform the unreliable service of IP into a reliable service?

Through the use of checksums (error detection), sequence numbers (for ordering and detecting duplicates), acknowledgments, timers, and retransmissions.

Question 149

What is the key difference between the Go-Back-N and Selective Repeat approaches to error recovery in a pipelined protocol?

GBN retransmits all packets from the lost packet onward upon a timeout or error. SR retransmits only the specific lost or corrupted packets.

Question 150

Why is pipelining necessary in high-speed networks?

Because the delay-bandwidth product can be very large, meaning a huge amount of data can be in transit. A stop-and-wait protocol would leave the link idle for long periods, leading to extremely low utilization.

Question 151

How does TCP's use of cumulative acknowledgments make it different from a pure Selective Repeat protocol?

In a pure SR protocol, each packet is individually acknowledged. In TCP, an ACK for sequence number n cumulatively acknowledges all data up to n . This means the sender cannot know exactly which packets have been received correctly if they are out of order, which is why it needs mechanisms like duplicate ACKs.

Question 152

What is the relationship between the send buffer, the receive buffer, the receive window (rwnd), and flow control?

The receive buffer size (RcvBuffer) and the amount of data already in it determine the spare room, which is advertised as rwnd to the sender. The sender uses rwnd to limit how much unacknowledged data it has in its send buffer, thus implementing flow control.

Question 153

Explain how the three-way handshake helps in initializing critical state information in both the client and server.

It allows both sides to agree on each other's initial sequence numbers (client_isn, server_isn). The client learns server_isn from the SYNACK, and the server knows the client has received its SYNACK when it gets the ACK for server_isn. This synchronizes the sequence number spaces for the connection.

Question 154

Why is fast retransmit more efficient than waiting for a timeout for retransmitting a lost segment?

Because the timeout interval is often significantly longer than the time it takes for duplicate ACKs to be generated. Fast retransmit allows for quicker recovery from single lost packets, reducing end-to-end delay.

Question 155

How does the exponential backoff of the timeout interval during retransmission serve as a primitive congestion control mechanism?

It causes the sender to slow down its retransmission rate dramatically when faced with persistent packet loss (a sign of congestion), giving the network time to clear the congestion.

Question 156

Under what condition might a TCP sender retransmit a segment that was not actually lost?

If the ACK for that segment is delayed longer than the timeout interval, leading to a premature timeout.

Question 157

How does a TCP receiver's behavior upon receiving an out-of-order segment differ from a GBN receiver's behavior?

A TCP receiver typically buffers the out-of-order segment, while a GBN receiver discards it.

Question 158

What is the "window" in the context of GBN and TCP, and what does it represent?

It is the range of sequence numbers that the sender is allowed to have outstanding (sent but not ACK'd) at any given time. It represents the pipeline's capacity.

Question 159

Why must the range of sequence numbers in a pipelined protocol be larger than in a stop-and-wait protocol?

To uniquely identify all the packets that could be in the pipeline at once, preventing ambiguity between new packets and retransmissions.

Question 160

How does the MSS relate to the MTU?

MSS is typically set to MTU minus the size of the TCP and IP headers to avoid fragmentation at the IP layer.

Question 161

What role do the source and destination port numbers play in a Web browsing session?

The destination port (typically 80 for HTTP) identifies the Web server process. The source port is a random ephemeral port chosen by the client, which, together with the client's IP address, allows the server to send responses back to the correct client process (the browser).

Question 162

In the context of TCP congestion control (mentioned briefly), why is it important for UDP-based applications to be designed carefully?

Because UDP has no built-in congestion control. A greedy UDP application could send data at a high rate, congest the network, and cause TCP connections to throttle back, potentially "starving" them.

Question 163

What is the significance of the initial sequence number being chosen at random?

To prevent security attacks where an attacker impersonates one of the connection endpoints by predicting the sequence numbers and injecting malicious data into the stream.

Question 164

How does the TCP sender's behavior upon receiving a duplicate ACK differ from its behavior upon receiving a cumulative ACK that moves the window forward?

A cumulative ACK that moves the window forward causes the sender to consider those packets delivered and to potentially send new data. A duplicate ACK does not move the window forward but, after three duplicates, triggers a fast retransmit.

Question 165

Describe a scenario where a cumulative acknowledgment prevents an unnecessary retransmission.

Suppose segments 1, 2, and 3 are sent. ACK 1 is lost, but ACK 2 and ACK 3 arrive. ACK 3, being cumulative, implicitly acknowledges that segment 1 was received. The sender, seeing ACK 3, knows segment 1 was successful and does not retransmit it, even though ACK 1 was lost.

Question 166

What is the purpose of the persist timer in TCP?

To handle the zero-window-size scenario. It triggers the sending of a window probe segment to prevent the deadlock described in the flow control section.

Question 167

How does the TCP receiver's advertised window size (rwnd) and the congestion window (cwnd, mentioned in later sections) both control the sender's transmission rate?

The sender's effective window is the minimum of rwnd (receiver's buffer limit) and cwnd (network's congestion limit). This ensures the sender is limited by both the receiver's capacity and the network's capacity.

Question 168

Why is the connection teardown process a four-step sequence with two FINs and two ACKs, rather than a three-way handshake?

Because a TCP connection is full-duplex. Each side must close its own half of the connection independently. One FIN/ACK pair closes one direction, and the second FIN/ACK pair closes the other direction.

Question 169

If a server process terminates abruptly, what might TCP on the server host send to any connected clients?

It would send a segment with the RST (reset) bit set to tear down the connection abnormally.

Question 170

Contrast the information used to demultiplex a segment in a connectionless (UDP) versus a connection-oriented (TCP) transport protocol.

UDP uses only the destination IP address and destination port number. TCP uses the combination of source IP address, source port number, destination IP address, and destination port number.

End of Study Guide

Wish you all the best!
Fathy.